

Linux

Administrator's Survival Guide

Essential Skills, Real Commands, Practical Examples and Troubleshooting for Junior Linux Administrators



System Administration



Users, Permissions and Security



Processes, Services and Logs



Networking and Troubleshooting



Storage, Memory and Performance



Real-World Scenarios and Practice



Learn. Practice. Troubleshoot. Build Real Skills.

A practical guide to help you understand, manage and troubleshoot Linux systems with confidence.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Your Linux Survival Workflow

Why a Workflow?

As a Linux administrator, you will often face problems such as services not working, servers being slow, or users unable to connect. A clear workflow helps you stay calm, avoid random commands, and find the real cause before making changes.



Junior Engineer Tip

Always inspect and understand the current state before making any changes on a server.

5-Step Linux Admin Workflow



Why Inspect Before Changing?

- You understand the current state of the server.
- You avoid making the problem worse.
- You can identify the real cause instead of guessing.
- You keep a record of what changed and why.
- This is a professional habit used in production environments.



Common Beginner Mistake

- Running random commands without understanding the system state.
- Making changes without checking the impact.
- Restarting services without knowing the real cause.
- Not verifying if the issue is actually fixed.

Essential Commands to Understand the Current State

Command	What it Does	Example Output (shortened)	When You Use It
<code>whoami</code>	Shows the current user account.	ubuntu	Confirm which user you are logged in as.
<code>hostname</code>	Shows the server hostname.	web-01	Identify which server you are working on.
<code>pwd</code>	Shows your current directory path.	/home/ubuntu	Confirm where you are before running commands.
<code>uptime</code>	Shows how long the server has been running and load average.	10:24:01 up 5 days, load average: 0.08, 0.15, 0.12	Check server uptime and current load.
<code>date</code>	Shows the current date and time.	Wed Oct 1 10:24:01 UTC 2026	Confirm the server time (e.g. for logs and troubleshooting).
<code>uname -r</code>	Shows the current kernel version.	5.15.0-102-generic	Check the kernel version for compatibility or support.



Real World Example

A user reports the application is slow. Before restarting anything, you check the current user, hostname, uptime and load average to understand the server's state. This helps you decide the next steps based on real evidence.



Practice This

Log in to a Linux server or VM and run each command above. Write down the output and explain what it tells you about your system.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Know What System You're On

Why This Matters?

Before troubleshooting or installing software, you need to know what system you are working on. Different Linux distributions, kernels, and architectures use different package managers, paths, and configurations. Knowing the environment helps you use the correct commands, follow the right documentation, and avoid making mistakes.



Junior Engineer Tip

Always identify the system details first when you connect to a new server. It saves time and prevents running the wrong commands.

1 Check Linux Distribution

Shows the distribution name and version.

```
ubuntu@linux:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
ID=ubuntu
ID_LIKE=debian
```

When you use it:

To know the exact Linux distribution and version (e.g. Ubuntu, Debian, CentOS, RHEL, Amazon Linux).

2 Check Kernel Version

Shows the current kernel version.

```
ubuntu@linux:~$ uname -r
5.15.0-102-generic
```

When you use it:

To confirm the kernel version for compatibility, troubleshooting or support.

3 Check System Architecture

Shows the CPU architecture.

```
ubuntu@linux:~$ arch
x86_64
```

When you use it:

To know if the system is 64-bit (x86_64) or 32-bit, which is important when installing software.

4 Check Hostname

Shows the server hostname.

```
ubuntu@linux:~$ hostnamectl
Static hostname: web-01
Icon name: computer-vm
Operating System: Ubuntu 22.04.4 LTS
Kernel: Linux 5.15.0-102-generic
```

When you use it:

To identify which server you are working on, especially in environments with many servers.

5 Check Uptime and Load

Shows how long the system has been running.

```
ubuntu@linux:~$ uptime
10:24:01 up 5 days, 2:14, 1 user,
load average: 0.08, 0.15, 0.12
```

When you use it:

To check server uptime and current load average.

6 Check Virtualization or Cloud Context

Shows hardware and virtualization details.

```
ubuntu@linux:~$ hostnamectl
Virtualization: kvm
Operating System: Ubuntu 22.04.4 LTS
Kernel: Linux 5.15.0-102-generic
Architecture: x86-64
```

When you use it:

To know if the server is a VM, physical server or running in a cloud environment.



Common Beginner Mistakes

- Using the wrong package manager for the distribution.
- Installing software for the wrong architecture.
- Following a guide for a different Linux version.
- Not checking the kernel version when troubleshooting.
- Assuming the server is physical when it is a cloud VM.



Key Takeaways

- Always identify the distribution, kernel and architecture before making changes.
- It helps you use the correct commands and documentation.
- It prevents errors and failed installations.
- Know if you are on a VM, physical server or cloud environment.
- Make this a habit every time you access a new server.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Find Your Way Around Linux

Why This Matters?

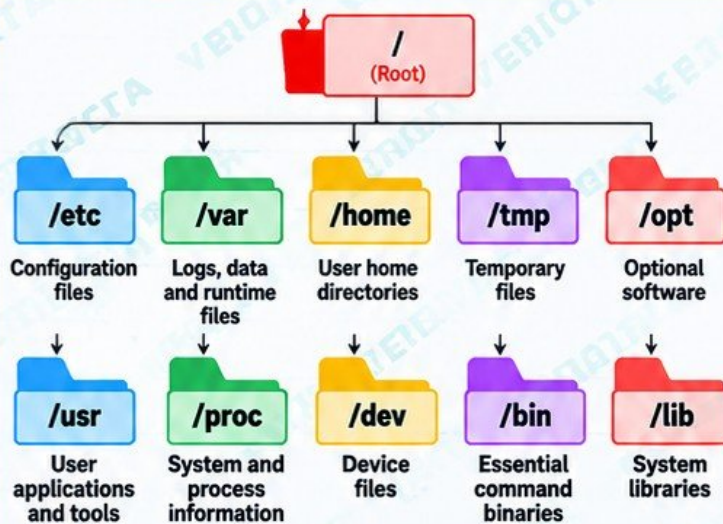
As a Linux administrator, you will constantly navigate the filesystem to find configuration files, logs, applications and data. Understanding the directory structure helps you know where to look, use the right commands and avoid making mistakes.



Junior Engineer Tip

Learn the purpose of each important directory. It saves you time and helps you know where to find files when troubleshooting.

Linux Filesystem Structure (Key Directories)



Common Directories Explained

Directory	Purpose / What You'll Find
/	Root of the entire filesystem.
/etc	Configuration files (e.g. nginx, ssh, systemd).
/var	Logs, databases, runtime files, spools and cached data.
/home	Home directories for regular users.
/tmp	Temporary files (usually cleaned on reboot).
/opt	Optional third-party software.
/usr	User applications, tools and libraries.
/proc	Virtual files with system and process information.
/dev	Device files (disks, terminals, etc).

Essential Navigation Commands

Command	What It Does	Example	Typical Output (shortened)
pwd	Shows your current directory path.	<code>ubuntu@linux:~\$ pwd</code>	<code>/home/ubuntu</code>
ls	Lists files and directories.	<code>ubuntu@linux:~\$ ls -l</code>	total 8 drwxr-xr-x 2 ubuntu ubuntu 4096 Oct 1 18:24 Documents
cd	Changes directory.	<code>ubuntu@linux:~\$ cd /etc</code>	(no output, path changes)
find	Searches for files or directories.	<code>ubuntu@linux:~\$ find /var -name "*.log" -type f</code>	<code>/var/log/syslog</code> <code>/var/log/auth.log</code>
realpath	Shows the absolute path of a file or directory.	<code>ubuntu@linux:~\$ realpath nginx.conf</code>	<code>/etc/nginx/nginx.conf</code>



Real World Examples

- Go to `/etc` to edit service configuration files.
- Check `/var/log` to troubleshoot application issues.
- Navigate to `/home/username` to access a user's files.
- Use `/tmp` for temporary files during troubleshooting.
- Find installed tools in `/usr/bin` or `/usr/local/bin`.
- Check device files in `/dev` when working with storage.



Common Beginner Mistakes

- Running commands from the wrong directory.
- Confusing `/` (root) with `/home`.
- Editing files in the wrong directory.
- Deleting files in `/etc` or `/var` without understanding.
- Using relative paths when an absolute path is needed.
- Assuming everything is in your home directory.



Practice This

- Use `pwd` to check your current directory.
- Navigate to `/etc` and list the contents.
- Find all `.conf` files under `/etc` using `find`.
- Use `realpath` to get the full path of a file.
- Explore `/var/log` and identify log files.
- Navigate back to your home directory.



Find the File You Need

Why This Matters?

As a Linux administrator, you will often need to find configuration files, logs, directories, or commands. Knowing the right tools and search methods saves time and helps you troubleshoot faster.



Junior Engineer Tip

Start with the fastest and safest method. Use `locate` or `which` for quick searches, and use `find` when you need more control.

Command	What It Does	Example	Typical Output (shortened)	When You Use It
find	Searches for files or directories based on name, type, size, etc.	<code>sudo find / -name "nginx.conf" 2>/dev/null</code>	<code>/etc/nginx/nginx.conf</code> <code>/usr/local/nginx/conf/nginx.conf</code>	When you need a detailed search or the file may not be indexed.
locate	Searches a database of files (faster).	<code>locate nginx.conf</code>	<code>/etc/nginx/nginx.conf</code> <code>/usr/share/doc/nginx/nginx.conf</code>	When you need a quick search (database must be updated).
which	Shows the full path of a command (executable).	<code>which nginx</code>	<code>/usr/sbin/nginx</code>	To find the executable used when you run a command.
command -v	Shows the path of a command (works in more environments).	<code>command -v docker</code>	<code>/usr/bin/docker</code>	To check if a command exists and where it is located.
whereis	Finds the binary, source and manual page locations.	<code>whereis ssh</code>	<code>ssh: /usr/bin/ssh</code> <code>/usr/share/man/man1/ssh.1.gz</code>	To find all related files for a command.



Common find Examples

- Find a file by name**
`sudo find / -name "app.conf" 2>/dev/null`
- Find a directory**
`sudo find / -type d -name "logs" 2>/dev/null`
- Find by file type**
`sudo find /var -type f -name "*.log"`
- Find files modified in last 7 days**
`sudo find / -name "*.conf" -mtime -7 2>/dev/null`
- Find large files (> 100MB)**
`sudo find / -type f -size +100M 2>/dev/null`



Search for Configuration and Log Files

- Find Nginx configuration**
`sudo find /etc -name "nginx.conf" 2>/dev/null`
- Find application logs**
`sudo find /var/log -type f -name "*.log"`
- Find a specific config in /etc**
`sudo find /etc -type f -name "*.conf"`
- Find files containing text**
`grep -r "error" /var/log 2>/dev/null`
- Combine find with grep**
`sudo find / -name "*.conf" -exec grep -l "port" {} \; 2>/dev/null`



Permission Denied Situations

- You may see "Permission denied" when searching system directories like `/root`, `/proc` or other protected paths.
- Use `sudo` to search as an administrator.
- Add `2>/dev/null` to hide permission error messages.
- Be careful when running `sudo find /` on production servers to avoid unnecessary load.



Common Beginner Mistakes

- Using `locate` before updating the database.
- Forgetting to use `sudo` when searching system directories.
- Searching the entire system without a specific path or name.
- Not using `2>/dev/null`, which shows many permission errors.
- Confusing `which` with `locate` (which only shows command paths).



Best Practices

- Use `locate` or `which` for quick searches.
- Use `find` with specific paths to save time and reduce load.
- Use `2>/dev/null` to hide permission errors.
- Be specific with file names to avoid too many results.
- Double-check the path before making any changes to important files.



Practice This

- Find the `ssh_config` file.
- Locate all `.log` files under `/var/log`.
- Find the path of the `docker` command.
- Search for a file named `app.conf` anywhere on the system.
- Use `grep` to find lines containing "error" in log files.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Inspect Files Without Breaking Them

Why This Matters?

As a Linux administrator, you should always inspect files before editing or making changes. This helps you understand the content, format, size and structure, prevents mistakes, and ensures you are working on the correct file.



Junior Engineer Tip

Inspect first, then edit. This helps you avoid deleting important content or breaking a working configuration.

1 Use cat to View Small Files

Displays the entire content of a file.

```
ubuntu@linux:~$ cat /etc/hostname
web-01
```

- Good for short files.
- Avoid using on very large files.

2 Use less to View Large Files

View file content one page at a time.

```
ubuntu@linux:~$ less /var/log/syslog
```

- Use arrow keys to navigate.
- Press **q** to quit.
- Best for large files like logs.

3 Use head to View the Beginning

Shows the first 10 lines (default).

```
ubuntu@linux:~$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

- Useful to quickly check file content.
- Use **-n** to specify the number of lines.

4 Use tail to View the End

Shows the last 10 lines (default).

```
ubuntu@linux:~$ tail /var/log/syslog
Oct 1 10:24:01 web-01 systemd[1]:
Started nginx.service.
```

- Useful for logs and real-time monitoring.
- Use **-f** to follow new lines as they are added.

5 Use grep to Search for Text

Finds lines that match a pattern.

```
ubuntu@linux:~$ grep "error" /var/log/syslog
Oct 1 10:25:01 web-01 app[123]: error:
failed to connect to database
```

- Searches for specific words or patterns.
- Use **-i** for case-insensitive search.
- Use **-n** to show line numbers.

6 Use wc to Check File Size and Lines

Counts lines, words and bytes.

```
ubuntu@linux:~$ wc /var/log/syslog
12345 54321 987654 /var/log/syslog
```

- First column: number of lines.
- Second column: number of words.
- Third column: number of bytes.

7 Use file to Identify File Type

Shows the file type and format.

```
ubuntu@linux:~$ file /etc/nginx/nginx.conf
/etc/nginx/nginx.conf: ASCII text
```

- Helps you know if a file is text, binary, script, etc.
- Useful when you are not sure what type of file it is.

8 Use stat to Get Detailed File Information

Shows detailed metadata about a file.

```
ubuntu@linux:~$ stat /etc/nginx/nginx.conf
Size: 2768    Blocks: 8    IO Block: 4096
Access: 2026-10-01 09:12:34
Modify: 2026-09-30 16:45:21
```

- Shows size, permissions, owner, timestamps and more.
- Useful for troubleshooting permission issues.



Best Practices

- Inspect a file before editing it.
- Use less for large files.
- Use grep to find specific information.
- Check file type and permissions when in doubt.
- Always work on the correct file.



Common Beginner Mistakes

- Using cat on very large files.
- Editing a file without checking its content.
- Searching the entire / directory without sudo.
- Not using grep to filter results.
- Modifying the wrong file.



Practice This

- 1 View /etc/hostname with cat.
- 2 Check the first 5 lines of /etc/passwd.
- 3 Find the word root in /etc/passwd.
- 4 Check the size of /var/log/syslog.
- 5 Get detailed info for /etc/nginx/nginx.conf.

Users, Groups & Identity

Why This Matters?

Linux uses users and groups to control access to files, processes and system resources. Understanding UID, GID and system accounts helps you manage permissions, improve security and troubleshoot issues. Many services run under dedicated identities to reduce risk and limit damage if the service is compromised.



Junior Engineer Tip

Do not run applications as root. Services should run under dedicated, limited user accounts with only the permissions they need.

1 Check Your Current Identity

Shows the current user and their groups.

```
ubuntu@linux:~$ whoami
ubuntu
ubuntu@linux:~$ id
uid=1000(ubuntu) gid=1000(ubuntu)
groups=1000(ubuntu),4(adm),24(cdrom)
```

- whoami shows the current username.
- id shows UID, GID and group memberships.
- UID uniquely identifies a user.

2 View All Logged In Users

Shows who is currently logged in to the system.

```
ubuntu@linux:~$ who
ubuntu :0 2026-10-01 10:24 (:0)
ubuntu@linux:~$ w
10:24:01 up 2:14, 1 user, load average: 0.08
USER TTY FROM LOGIN@ IDLE JCPU PCPU
ubuntu :0 :0 10:24 ?xdm? 0.12s 0.05s
```

- who shows current logged in users.
- w shows more detail including what they are doing.

3 View User Information from /etc/passwd

Shows user account details stored in /etc/passwd.

```
ubuntu@linux:~$ cat /etc/passwd | grep ubuntu
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
```

- Fields: username : password : UID : GID : comment : home directory : default shell.
- System users usually have low UIDs (e.g. 1-999).

4 View Group Information from /etc/group

Shows group details stored in /etc/group.

```
ubuntu@linux:~$ cat /etc/group | grep sudo
sudo:x:27:ubuntu
```

- Shows group name, GID and group members.
- Users can belong to multiple groups.
- Groups control access to files, directories and system resources.

5 Use getent for Complete Information

Works with local and centralized user databases.

```
ubuntu@linux:~$ getent passwd ubuntu
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
ubuntu@linux:~$ getent group sudo
sudo:x:27:ubuntu
```

- getent checks all configured sources (local or LDAP).
- Useful on systems using directory services.

6 Create a New User

Creates a new user account.

```
sudo useradd -m -s /bin/bash devuser
```

- -m creates a home directory.
- -s sets the default shell.
- Always follow your organization's naming standards.

Add User to a Group

```
sudo usermod -aG sudo devuser
```

- -aG adds the user to an existing group.
- The user must log out and back in for changes to apply.

7 Modify an Existing User

Changes user properties.

```
sudo usermod -s /bin/zsh devuser # Change shell
sudo usermod -L devuser # Lock the user
sudo usermod -U devuser # Unlock the user
```

- You can also change the home directory, comment or groups.
- Use with care, especially on production systems.

8 Why Services Use Dedicated Identities

- Limits the damage if the service is compromised.
- Restricts access to only what the service needs.
- Improves auditing and accountability.
- Prevents accidental changes to system files.
- Avoids running everything as root.



Common Beginner Mistakes

- Running applications as root.
- Giving unnecessary sudo access.
- Deleting or locking the wrong user.
- Not understanding which user a service runs as.



Key Takeaways

- Every user has a unique UID and GID.
- Use dedicated service accounts.
- Understand /etc/passwd and /etc/group.
- Use getent on systems with directory services.
- Manage users and groups safely and carefully.



Practice This

- 1 Find your current user and groups.
- 2 List all logged in users.
- 3 Locate a user in /etc/passwd.
- 4 Create a new user and add to a group.
- 5 Change the shell of a user.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Permissions Without chmod 777

Why This Matters?

Linux uses permissions to control who can read, write and execute files and directories. Understanding permissions helps you secure the system, prevent accidental changes and fix access issues without using dangerous permission settings like chmod 777.



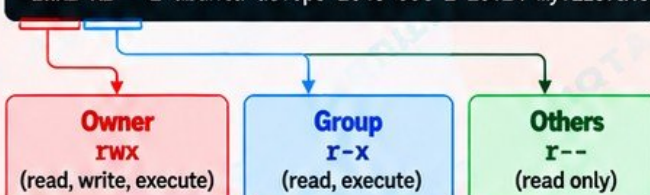
Junior Engineer Tip

Never use chmod 777. It gives full access to everyone and creates a serious security risk.

1 Understand rwx Permissions

Each file has permissions for owner, group and others.

```
ubuntu@linux:~$ ls -l myfile.txt
-rwxr-xr-- 1 ubuntu devops 2048 Oct 1 10:24 myfile.txt
```



2 Numeric Permission Modes

Permissions can also be represented using numbers.

Permission	Symbol	Number
Read (r)	r--	4
Write (w)	-w-	2
Execute (x)	--x	1

Common Examples

chmod 755	rwxr-xr-x	7 5 5
chmod 644	rw-r--r--	6 4 4

3 Change Permissions with chmod

Modify file or directory permissions.

```
ubuntu@linux:~$ chmod 755 script.sh # rwxr-xr-x
ubuntu@linux:~$ chmod 644 file.txt # rw-r--r--
ubuntu@linux:~$ chmod +x script.sh # add execute
ubuntu@linux:~$ chmod u-w file.txt # remove write
```

- Use numeric mode (e.g. 755) or symbolic mode (e.g. u+w).
- Only give the permissions that are required.

4 Change Ownership with chown and chgrp

Change the owner or group of a file or directory.

```
ubuntu@linux:~$ sudo chown ubuntu file.txt
ubuntu@linux:~$ sudo chown ubuntu:devops file.txt
ubuntu@linux:~$ sudo chgrp devops file.txt
```

- chown changes the owner.
- chgrp changes the group.
- Use sudo when you do not own the file.

5 Default Permissions with umask

umask controls the default permissions for new files.

```
ubuntu@linux:~$ umask # shows current setting
0022
ubuntu@linux:~$ umask 027 # set a new default
```

- New files will have permissions based on the umask value.
- Lower umask means more permissions, higher means less.
- Common values: 022 (files 644, directories 755).

6 Executable Permissions

A file must have execute permission to run as a program.

```
ubuntu@linux:~$ ls -l script.sh
-rw-r--r-- 1 ubuntu ubuntu 123 Oct 1 10:24 script.sh
ubuntu@linux:~$ chmod +x script.sh
ubuntu@linux:~$ ./script.sh # now it can run
```

- Use chmod +x to make a file executable.
- Check the shebang line (e.g. #!/bin/bash).
- Ensure the file is owned by the correct user.

7 Troubleshoot Permission Denied

Common causes and how to fix them.

Cause	Example	Fix
No execute permission	bash: ./script.sh: Permission denied	chmod +x script.sh
Wrong owner	Permission denied (editing file)	sudo chown user:file
Directory lacks execute permission	cd: Permission denied	chmod +x /path/to/dir
Access restricted to root	Permission denied (/etc file)	sudo or adjust permissions safely

8 Common Beginner Mistakes

- Using chmod 777 on files or directories.
- Giving more permissions than required.
- Not understanding the difference between owner, group and others.
- Forgetting to set execute permission on scripts.
- Changing ownership without understanding the impact.
- Ignoring umask settings.



Practice This

- Check the permissions of /etc/passwd.
- Make a script file executable and run it.
- Create a new file with specific permissions (640).
- Change the owner and group of a file.
- Fix a "Permission denied" error in a real scenario.

Use sudo Without Creating Risk

Why This Matters?

sudo gives normal users the ability to run commands with higher privileges. Using it correctly improves security, keeps an audit trail, and reduces the risk of accidental damage. Running everything as root is dangerous and can lead to system-wide changes, security issues and data loss.



Junior Engineer Tip

Use sudo only when needed. Always know what command you are running and why. Do not stay logged in as root for daily work.

1 Root vs Normal Users

Check your current user and switch to root (for comparison).

```
ubuntu@linux:~$ whoami
ubuntu
ubuntu@linux:~$ sudo whoami
root
```

- Normal users have limited privileges.
- Root has full access to the system.
- Only use root or sudo when necessary.

2 Run a Single Command with sudo

Use `sudo` to run a command with higher privileges.

```
ubuntu@linux:~$ sudo apt update
[sudo] password for ubuntu:
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
```

- You will be asked for your password.
- Only that command runs with elevated privileges.
- Your normal user session remains unchanged.

3 See What You Can Run

List the commands your user is allowed to run with sudo.

```
ubuntu@linux:~$ sudo -l
Matching Defaults entries for ubuntu on linux:
env_reset, mail_badpass,
User ubuntu may run the following commands on linux:
(root) /usr/bin/apt, /usr/bin/systemctl
```

- Shows which commands you can run with sudo.
- Helps you understand your permissions.
- Useful for troubleshooting permission issues.

4 Configure sudo Safely

Edit the sudoers file using `visudo` (never edit directly).

```
ubuntu@linux:~$ sudo visudo
```

- `visudo` opens `/etc/sudoers` safely.
- It checks for syntax errors before saving.
- Always use `visudo` instead of editing the file directly to avoid breaking sudo access.

Example: Allow a user to run specific commands

```
ubuntu ALL=(root) /usr/bin/systemctl, /usr/bin/apt
```

5 Common sudoers Examples

Examples of useful and safe configurations.

Purpose	Example (in <code>/etc/sudoers</code>)
Allow full access (admin user)	ubuntu ALL=(ALL:ALL) ALL
Allow specific command only	ubuntu ALL=(root) /usr/bin/systemctl restart nginx
Allow group members	%devops ALL=(root) /usr/bin/apt
Allow without password (use with care)	ubuntu ALL=(ALL) NOPASSWD: /usr/bin/systemctl

6 Use sudo for Administrative Tasks

Common real-world examples.

```
sudo systemctl restart nginx
sudo apt install httpd
sudo journalctl -u nginx -n 50
sudo useradd devuser
sudo chown -R devuser:devops /opt/app
```

- Use sudo for system administration tasks.
- Always verify the command before running it.
- Use the minimum privileges required.

7 Why Running Everything as Root Is Dangerous

- A simple mistake can delete or overwrite critical files.
- Malware and compromised applications get full access.
- No clear audit trail of what was done.
- Increases the risk of security breaches.
- Harder to identify who made changes.
- Can break the system and cause downtime.

8 Best Practices

- Use sudo only when necessary.
- Prefer specific commands instead of full root access.
- Do not share your password.
- Use groups to manage access.
- Keep an audit trail (logs show sudo usage).
- Avoid using `NOPASSWD` unless there is a strong reason.



Common Beginner Mistakes

- Running every command with `sudo`.
- Editing `/etc/sudoers` directly instead of using `visudo`.
- Giving unnecessary full root access.
- Not understanding what a command does.



Key Takeaways

- `sudo` gives controlled elevated access.
- Use `visudo` to edit sudoers safely.
- Follow the principle of least privilege.
- Understand your permissions with `sudo -l`.
- Avoid running everything as root.



Practice This

- 1 Check your current user with `whoami`.
- 2 Run `sudo -l` and review the output.
- 3 Open `/etc/sudoers` with `visudo` (read only).
- 4 Allow a user to run a specific command.
- 5 Check sudo logs in `/var/log/auth.log`.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Understand What Is Running

Why This Matters?

Processes are running programs. Understanding what is running helps you troubleshoot issues, find the cause of high resource usage, connect processes to applications and services, and keep your system healthy and stable.



Junior Engineer Tip

When a system is slow or behaving strangely, check what is running before making any changes.

1 View Running Processes with ps

Shows a snapshot of current processes.

```
ubuntu@linux:~$ ps aux | head
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT   COMMAND
root         1   0.0  0.1  16884  1096 ?        Ss      /sbin/init
root      412   0.0  0.0   22608  1024 ?        Ss      /lib/systemd/systemd
ubuntu  1234   0.1  0.2   29180  5160 pts/0    S+      bash
ubuntu  5678   0.0  0.1   21732  3072 pts/0    R+      ps aux
```

- ps shows all running processes.
- Key columns: PID, PPID, CPU, MEM, STAT, COMMAND.
- Use ps aux or ps -ef for more details.

2 Find a Specific Process with pgrep

Find processes by name.

```
ubuntu@linux:~$ pgrep nginx
2345
2360
```

- Returns the PID of matching processes.
- Use -l to also show the process name.
- Useful for scripts and quick checks.

3 Monitor Processes with top

Real-time view of running processes.

```
ubuntu@linux:~$ top
PID      USER     PR  NI  VIRT  RES  %CPU %MEM     COMMAND
2345     root      20   0   162m  12m   5.3  0.6    nginx
5678     ubuntu    20   0   312m  45m   2.1  2.3    python
9012     ubuntu    20   0    98m   8m    1.0  0.4    node
```

- Shows real-time CPU, memory and process information.
- Press q to quit.
- Useful for identifying high resource usage.

4 See the Process Tree with pstree

Shows the parent and child relationship between processes.

```
ubuntu@linux:~$ pstree -p | head
systemd(1)─nginx(2345)─nginx(2360)
              │
              └─sshd(1023)─sshd(1200)─bash(1201)
                  │
                  └─cron(900)
```

- Shows the hierarchy (parent and child processes).
- Helps you understand which process started another.
- Useful when troubleshooting services.

5 Manage Foreground and Background Jobs

Run commands in the foreground or background.

```
ubuntu@linux:~$ sleep 300 &
[1] 6789
ubuntu@linux:~$ jobs
[1]  Running                  sleep 300
```

- Use & to run a command in the background.
- Use jobs to list background jobs.
- Use fg %1 to bring a job to the foreground.
- Use bg %1 to continue a stopped job in the background.

6 See Which Files a Process Uses

Use lsof to see open files and network connections.

```
ubuntu@linux:~$ lsof -p 2345 | head
COMMAND PID  USER  FD  TYPE DEVICE SIZE/OFF  NODE NAME
nginx   2345 root   cwd  DIR   8,1    4096     2  /
nginx   2345 root  rtd  DIR   8,1    4096     2  /
nginx   2345 root  txt  REG   8,1  1803128  411 /usr/sbin/nginx
```

- Shows files, sockets and network connections used by a process.
- Useful to understand what a process is accessing.
- Helps in troubleshooting and security investigations.

7 Common Process States

What the STAT column means.

State	Meaning	Description
R	Running	Currently using CPU
S	Sleeping	Waiting for an event
D	Uninterruptible	Waiting for disk or I/O
T	Stopped	Stopped by a signal
Z	Zombie	Process has finished but parent has not collected it

8 Connect Processes to Services

Find which process is running a service.

```
ubuntu@linux:~$ systemctl status nginx
● nginx.service - A high performance ...
   Loaded: loaded (/lib/systemd/system/nginx.service;
   Active: active (running) since Thu 2026-10-01
   Main PID: 2345 (nginx)
```

- Shows the process ID and service details.
- Helps you link a process to the application it runs.
- Useful when checking if a service is running correctly.



Common Beginner Mistakes

- Not checking what is running before making changes.
- Killing the wrong process.
- Not understanding the process tree.
- Ignoring background processes.
- Not checking open files and network connections.



Practice This

- 1 List all running processes.
- 2 Find the PID of a specific process.
- 3 View the process tree for a service.
- 4 Run a command in the background.
- 5 Use lsof to see open files for a process.

Stop a Process Safely

Why This Matters?

Processes should be stopped safely to avoid data loss, corruption and service issues. Understanding Linux signals helps you choose the right method, terminate the correct process, and verify that it actually stopped.



Junior Engineer Tip

Always try a graceful stop (SIGTERM) first. Use SIGKILL only when the process does not respond or is stuck.

1 Understand Process Signals

Signals tell a process to take a specific action.

Signal	Number	Meaning	When to Use
SIGTERM	15	Graceful stop (default)	Use first. Allows the process to clean up and exit.
SIGKILL	9	Force stop	Use only when the process does not respond to SIGTERM.
SIGHUP	1	Reload configuration	Common for services to reload config.
SIGINT	2	Interrupt (from keyboard)	Stop a foreground process (Ctrl + C).

2 Find the Process You Want to Stop

Identify the correct process before terminating it.

```
ubuntu@linux:~$ ps aux | grep nginx
```

```
root      2345  0.0  0.1 12340 456 ?        Ss   10:20 nginx
ubuntu    5678  0.0  0.0  6420 220 pts/0    S+   10:25 grep nginx
```

- Use ps aux, pgrep or top to find the process.
- Confirm you have the correct PID and process name.
- Check what the process is (application, service, script).

3 Stop a Process with SIGTERM (Recommended)

Ask the process to stop gracefully.

```
ubuntu@linux:~$ kill 2345
# same as: kill -15 2345
ubuntu@linux:~$ pkill nginx
# sends SIGTERM to all nginx processes
```

- Allows the process to finish current tasks.
- Gives the process time to clean up.
- Best practice for most situations.

4 Force Stop a Process with SIGKILL

Use this only when the process does not respond.

```
ubuntu@linux:~$ kill -9 2345
# or
ubuntu@linux:~$ pkill -9 nginx
```

- Immediately terminates the process.
- Does not allow cleanup (can cause data loss).
- Use only when SIGTERM fails or the process is stuck.
- Common for unresponsive or hung processes.

5 Verify That the Process Stopped

Check that the process is no longer running.

```
ubuntu@linux:~$ ps aux | grep nginx
ubuntu@linux:~$ pgrep nginx
# no output means it has stopped
```

- Run the same command you used to find the process.
- No output means the process has stopped.
- If it is still running, check if you used the correct PID or if it restarted automatically (e.g. a service).

6 Stop Processes by Name or Pattern

Useful when you do not know the exact PID.

```
ubuntu@linux:~$ pkill -f myapp
ubuntu@linux:~$ pkill -u ubuntu python
ubuntu@linux:~$ killall myapp
```

- pkill matches by process name or pattern.
- -f matches the full command line.
- -u stops processes owned by a specific user.
- killall stops all processes with the given name (use carefully).

7 Real-World Example

A process is using high CPU and needs to be stopped.

```
ubuntu@linux:~$ ps aux | grep myapp
ubuntu@linux:~$ kill 2345 # try SIGTERM
ubuntu@linux:~$ sleep 5
ubuntu@linux:~$ ps aux | grep myapp
# if still running:
ubuntu@linux:~$ kill -9 2345 # force stop
```



Common Beginner Mistakes

- Using SIGKILL immediately.
- Killing the wrong process.
- Not verifying that the process actually stopped.
- Not understanding what the process does.
- Stopping a process that is managed by a service (it may restart).
- Using killall or pkill with a broad pattern without checking.



Key Takeaways

- Find the correct process first.
- Use SIGTERM (kill or pkill) before SIGKILL.
- Verify that the process has stopped.
- Understand the impact before stopping.
- Be careful with killall and pkill patterns.



Practice This

- 1 Find a running process and stop it with SIGTERM.
- 2 Verify that it stopped.
- 3 Start a process and stop it with pkill.
- 4 Identify an unresponsive process and stop it with SIGKILL.
- 5 Check that the correct process actually stopped.

Manage Services with systemd

Why This Matters?

systemd manages services, startup, dependencies and system resources. Understanding and managing services is essential to keep applications running, ensure they start automatically after a reboot, and to troubleshoot issues when services fail.



Junior Engineer Tip

Always check the service status and logs before making changes. Understand what a service does and its dependencies.

1 Understand systemd Units

Units are configuration files that tell systemd how to manage a service.

```
ubuntu@linux:~$ ls /etc/systemd/system/
nginx.service  prometheus.service  myapp.service
```

- Common unit types: service, socket, target, timer, mount. Service units manage long-running applications.
- Unit files are located in /etc/systemd/system/ or /usr/lib/systemd/system/.
- Each service can have dependencies on other services.

3 Manage Services with systemctl

Common commands to control a service.

```
ubuntu@linux:~$ sudo systemctl start nginx # start service
ubuntu@linux:~$ sudo systemctl stop nginx # stop service
ubuntu@linux:~$ sudo systemctl restart nginx # restart service
ubuntu@linux:~$ sudo systemctl reload nginx # reload configuration
ubuntu@linux:~$ sudo systemctl enable nginx # start on boot
ubuntu@linux:~$ sudo systemctl disable nginx # disable on boot
```

- start: starts the service.
- stop: stops the service.
- restart: stops and starts the service.
- reload: re-reads configuration without stopping (if supported).
- enable: starts the service automatically at boot.
- disable: prevents the service from starting at boot.

5 View Logs and Troubleshoot

Check logs when a service fails to start.

```
ubuntu@linux:~$ sudo journalctl -u nginx -n 20
Oct 01 10:21:15 linux systemd[1]: Starting nginx...
Oct 01 10:21:15 linux nginx[2345]: nginx: [emerg] bind() to 0.0.0.0:80 failed (Address already in use)
Oct 01 10:21:15 linux systemd[1]: nginx.service: control process exited, code=exited, status=1/FAILURE
Oct 01 10:21:15 linux systemd[1]: Failed to start nginx.
```

- Use journalctl -u <service> to view recent logs.
- Look for error messages and the reason for failure.
- Check if required dependencies are running.
- Verify configuration files for errors.



Common Beginner Mistakes

- Not checking service status before making changes.
- Using restart when reload is enough.
- Ignoring logs when a service fails.
- Not understanding service dependencies.
- Enabling a service that is not configured correctly.



Key Takeaways

- Use systemctl to manage services.
- Know the difference between restart and reload.
- Always check logs when a service fails.
- Understand and verify dependencies.
- Enable critical services to start on boot.

2 Check Service Status

View the current state of a service.

```
ubuntu@linux:~$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled)
   Active: active (running) since Thu 2026-10-01 10:20:15
     Main PID: 2345 (nginx)
        Tasks: 3 (limit: 4621)
       Memory: 5.8M
      CGroup: /system.slice/nginx.service
              └─2345 nginx: master process /usr/sbin/nginx
                 2346 nginx: worker process
```

- Shows if the service is active, inactive or failed.
- Displays the main PID, uptime, memory usage and more.

4 Restart vs Reload

Understand the difference.

Action	What It Does	When to Use
restart	Stops and starts the service.	Use when configuration changes require a full restart or when the service is unstable.
reload	Re-reads the configuration without stopping (the service keeps running).	Use for applying configuration changes without downtime (if the service supports reload).



Not all services support reload. If reload is not supported, you may need to use restart.

6 Check Dependencies

See what other services a unit depends on.

```
ubuntu@linux:~$ systemctl list-dependencies nginx
● nginx.service
   └─network.target
      └─remote-fs.target
         └─sysinit.target
```

- Shows required and optional dependencies.
- A service may fail if a required dependency is not running.
- Use this to understand startup order and fix dependency issues.

7 Real-World Troubleshooting Example

Nginx fails to start due to port already in use.

```
ubuntu@linux:~$ sudo systemctl start nginx
Job for nginx.service failed.
ubuntu@linux:~$ sudo journalctl -u nginx -n 5
nginx: [emerg] bind() to 0.0.0.0:80 failed (Address already in use)
ubuntu@linux:~$ sudo ss -tln
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
apache2 1234 root 4u IPv4 12345 0t0 TCP *:http (LISTEN)
```

- Identify which process is using the port, stop it or change the configuration, then start the service again.



Practice This

- Check the status of a service.
- Start, stop, restart and reload a service.
- Enable a service to start on boot.
- View logs for a failed service.
- Troubleshoot a service that refuses to start.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Logs Tell You What Happened



Why This Matters?

Logs record what happens on your system. They help you find the cause of problems, understand system behavior, troubleshoot services and applications, and provide evidence instead of guessing.



Junior Engineer Tip

When something goes wrong, always check the logs first. Logs show the real evidence and save you time.

1 Common Log Locations

Important directories and files.

Path	What It Contains
/var/log	Application and system logs
/var/log/syslog	General system logs (Debian/Ubuntu)
/var/log/messages	General system logs (RHEL/CentOS)
/var/log/auth.log	Authentication and sudo logs
/var/log/nginx/	Nginx access and error logs
/var/log/apache2/	Apache access and error logs
/var/log/secure	Security related logs (RHEL/CentOS)
/var/log/kern.log	Kernel logs (Debian/Ubuntu)

2 View Logs with journalctl

journalctl is used to view logs from systemd and services.

```
ubuntu@linux:~$ journalctl -xe
ubuntu@linux:~$ journalctl -u nginx
ubuntu@linux:~$ journalctl -u nginx -n 50
ubuntu@linux:~$ journalctl -b
ubuntu@linux:~$ journalctl -u nginx --since "1 hour ago"
ubuntu@linux:~$ journalctl -u nginx --since "2026-10-01"
ubuntu@linux:~$ journalctl -p err -u nginx
```

- -xe shows detailed logs with explanations.
- -u filters logs for a specific service.
- -b shows logs for the current boot.
- --since filters logs by time.
- -p filters by priority (e.g. err, warning, info).

3 View and Filter Log Files

Use standard tools to read and search log files.

```
ubuntu@linux:~$ tail -n 50 /var/log/syslog
ubuntu@linux:~$ less /var/log/syslog
ubuntu@linux:~$ grep -i "error" /var/log/syslog
ubuntu@linux:~$ grep -i "nginx" /var/log/nginx/error.log
ubuntu@linux:~$ grep "Oct 01" /var/log/syslog
```

- Use tail to see the latest entries.
- Use less to view logs safely.
- Use grep to search for keywords, errors or a service name.
- Filter by date, time or term to find relevant information.

4 Kernel Logs with dmesg

dmesg shows kernel messages from the current boot.

```
ubuntu@linux:~$ dmesg
ubuntu@linux:~$ dmesg | grep -i error
ubuntu@linux:~$ dmesg | grep -i "disk"
ubuntu@linux:~$ dmesg --level=err,warn
```

- Shows hardware, driver and kernel related messages.
- Useful for troubleshooting boot issues, device errors and hardware problems.
- Use --level to filter by log level.

5 Filter Logs by Time and Priority

Find logs within a specific time range or priority level.

```
ubuntu@linux:~$ journalctl --since "2026-10-01 10:00" \
--until "2026-10-01 12:00"
ubuntu@linux:~$ journalctl -u nginx --since "1 hour ago"
ubuntu@linux:~$ journalctl -p warning
ubuntu@linux:~$ journalctl -p err -u ssh
```

- Use --since and --until to filter by time.
- Use -p to filter by priority (emerg, alert, crit, err, warning, notice, info, debug).
- Useful for finding what happened during a specific incident.

6 Real-World Troubleshooting Example

Nginx service refuses to start. Use logs to find the cause.

```
ubuntu@linux:~$ sudo systemctl status nginx
ubuntu@linux:~$ sudo journalctl -u nginx -n 50
ubuntu@linux:~$ sudo tail -n 50 /var/log/nginx/error.log
ubuntu@linux:~$ sudo dmesg | grep -i nginx
```

- 1 Check the service status for a quick overview.
- 2 View the service logs with journalctl.
- 3 Check the application log file (e.g. /var/log/nginx/error.log).
- 4 Check kernel logs if it seems like a system or hardware issue.
- 5 Look for the actual error message and fix the root cause.

7 Common Log Levels

Log priorities (lowest to highest severity).

Level	Description
emerg	System is unusable
alert	Action must be taken immediately
crit	Critical conditions
err	Error conditions
warning	Warning conditions
notice	Normal but significant
info	Informational messages
debug	Detailed debug information

8 Common Beginner Mistakes

- Not checking logs before making changes.
- Looking at the wrong log file.
- Ignoring error messages.
- Not filtering by time (too many logs).
- Not understanding log levels.
- Relying on guesses instead of actual evidence in the logs.
- Forgetting to check both application and system logs.



Key Takeaways

Logs show the real evidence of what happened. Use journalctl, log files and dmesg. Filter by time, unit and priority. Look for error messages and patterns. Always check logs before making changes.



Practice This

- 1 View logs for a running service.
- 2 Find error messages in logs.
- 3 Filter logs for a specific time range.
- 4 Check kernel logs with dmesg.
- 5 Troubleshoot a failed service using logs.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Is the Server Running Out of Disk?



Why This Matters?

Running out of disk space can cause applications to fail, services to stop, logs to stop writing, and even prevent the system from functioning properly. You should know how to check disk usage, find the real cause, and fix the issue safely.



Junior Engineer Tip

Do not immediately delete files when the disk is full. First find out what is using the space and whether it is files or inodes. This helps you avoid deleting important data.

1 Check Disk Space with df -h

Shows filesystem usage and available space.

```
ubuntu@linux:~$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	45G	3.2G	94%	/
tmpfs	2.0G	0	2.0G	0%	/dev/shm
/dev/sdb1	200G	120G	80G	60%	/data

- Use% shows how full the filesystem is.
- The Mounted on column shows where it is used.
- Focus on filesystems with high usage (e.g. 90%+).

2 Check Inode Usage with df -i

Shows inode usage (number of files).

```
ubuntu@linux:~$ df -i
```

Filesystem	Inodes	IUsed	IFree	IUse%	Mounted on
/dev/sda1	3276800	3200000	76800	98%	/
tmpfs	524288	400	523888	1%	/dev/shm
/dev/sdb1	13107200	800000	12307200	7%	/data

- Inodes are used for files and directories.
- You can run out of inodes even if you have free disk space.
- If IUse% is high, you have too many files.

3 Find Large Files and Directories with du

Find what is using the most space.

```
ubuntu@linux:~$ sudo du -h /var | sort -hr | head -n 10
```

5.2G	/var/log
2.1G	/var/log
800M	/var/cache
300M	/var/tmp
50M	/var/spool

- du shows actual disk usage by file and directory.
- Use `sort -hr` to see the largest items first.
- Start with top level directories (e.g. /var, /home).

4 View Disks and Mounts

See available disks, partitions and mount points.

```
ubuntu@linux:~$ lsblk
```

NAME	MAJ:MIN	SIZE	TYPE	MOUNTPOINTS
sda	8:0	50G	disk	
└─sda1	8:1	50G	part	/
sdb	8:16	200G	disk	
└─sdb1	8:17	200G	part	/data

```
ubuntu@linux:~$ findmnt
```

TARGET	SOURCE	FSTYPE	OPTIONS
/	/dev/sda1	ext4	rw,relatime
/data	/dev/sdb1	ext4	rw,relatime
/bott	/dev/sda2	ext4	rw,relatime

- `lsblk` shows your disks and partitions.
- `findmnt` shows mounted filesystems and options.
- Use this to understand where each filesystem is mounted.

5 Diagnose "No Space Left on Device"

Steps to find the real cause.

- 1 Check disk usage: `df -h`
- 2 Check inode usage: `df -i`
- 3 Find large directories: `sudo du -h / | sort -hr | head`
- 4 Check for many small files: `find /path -type f | wc -l`
- 5 Check open deleted files: `sudo lsof | grep deleted`
- 6 Check logs and application data.
- 7 Decide the safe action (rotate logs, clean cache, move data, or increase disk).

6 Mount a Filesystem (Example)

Mount a filesystem manually or check mount options.

```
ubuntu@linux:~$ sudo mount /dev/sdb1 /data
ubuntu@linux:~$ mount | grep /data
/dev/sdb1 on /data type ext4 (rw,relatime)
```

- Use `mount` to mount a filesystem.
- Check mount options to understand how it is configured.
- Use `umount` to safely unmount when needed.

7 Common Causes of Disk Full

- Large log files (e.g. /var/log).
- Application data growing.
- Unbounded container or service logs.
- Too many small files (inode exhaustion).
- Large core dumps.
- Leftover temporary files (/tmp).
- Backup files or old releases.

8 Common Beginner Mistakes

- Immediately deleting files without checking.
- Ignoring inode usage.
- Not checking which filesystem is full.
- Deleting important application data.
- Not investigating why the disk filled up.
- Forgetting to rotate or clean logs.
- Not checking for open deleted files.



Key Takeaways

Always check disk and inode usage. Find the real cause before deleting. Use `df`, `du`, `lsblk` and `findmnt`. Check logs, application data and open files. Plan a long-term solution (cleanup, log rotation or increase disk size).



Practice This

- 1 Check disk and inode usage on your server.
- 2 Find the largest directories in /var.
- 3 Identify what is using the most inodes.
- 4 Locate and investigate a "No space left on device" error.
- 5 Clean up logs safely and verify the space is recovered.

Is Memory Actually the Problem?

Why This Matters?

Memory issues can cause slow performance, application failures and even the system to become unresponsive. Understanding how Linux uses memory helps you identify real memory problems, avoid unnecessary actions, and choose the right solution.



Junior Engineer Tip

Low free memory does not automatically mean Linux is out of memory. Linux uses free memory for cache and buffers to improve performance.

1 Check Memory Usage with free -h

Shows total, used, free, cache, buffers and available memory.

```
ubuntu@linux:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6G	6.1G	200M	400M	1.3G	1.0G
Swap:	2.0G	512M	1.5G			

- available is a better indicator than free.
- cache and buff/cache are used for performance.
- Swap shows the space used when RAM is full.

2 Understand What Each Field Means

Memory fields explained.

Field	Meaning
total	Total physical memory (RAM).
used	Memory currently in use (including cache).
free	Memory not used for anything.
buff/cache	Memory used for file cache and buffers (can be freed if needed).
available	Memory available for new applications (best indicator).
swap	Disk space used when RAM is full.

3 Monitor Memory in Real Time with top

Shows running processes and memory usage.

```
ubuntu@linux:~$ top
```

PID	USER	PR	NI	VIRT	RES	%MEM	%CPU	COMMAND
2345	root	20	0	1.2g	512m	6.5	2.1	java
5678	ubuntu	20	0	800m	300m	3.8	1.5	python
9012	ubuntu	20	0	600m	250m	2.9	0.8	nginx

- Press **M** to sort by memory usage.
- Identify processes using the most RAM.
- Check if high memory usage is expected for the application.

4 Use vmstat for Memory and Swap Activity

Shows memory, swap and I/O in real time.

```
ubuntu@linux:~$ vmstat 1 5
```

procs				-----memory-----				---swap---		-----io----	
r	b	swpd	free	buff	cache	si	so	bi	bo		
1	0	512000	200000	300000	1200000	0	0	5	10		
0	0	512000	180000	320000	1210000	0	0	2	4		
0	0	512000	160000	330000	1215000	0	0	1	3		

- swpd shows swap used.
- si and so show swap in and swap out.
- High si/so values mean the system is under memory pressure.

5 Check Detailed Memory Info in /proc/meminfo

Provides detailed memory statistics.

```
ubuntu@linux:~$ grep -E 'MemTotal|MemFree|MemAvailable|SwapTotal|SwapFree' /proc/meminfo
```

```
MemTotal:      8150940 kB
MemFree:       205432 kB
MemAvailable:  1043528 kB
SwapTotal:    2097148 kB
SwapFree:     1572860 kB
```

- MemAvailable is the most useful value.
- SwapTotal and SwapFree show swap configuration.
- Use this when you need more detailed information.

6 Find Memory-Heavy Processes with ps

Shows processes sorted by memory usage.

```
ubuntu@linux:~$ ps aux --sort=-%mem | head -n 10
```

USER	PID	%CPU	%MEM	VSZ	RSS	COMMAND
root	2345	2.1	6.5	1.2g	512m	java
ubuntu	5678	1.1	3.8	800m	300m	python
ubuntu	9012	0.8	2.9	600m	250m	nginx

- --sort=-%mem sorts by memory usage (highest first).
- Helps you find which application is using the most RAM.
- Use with top for real time monitoring.

7 Common Causes of High Memory Usage

- Memory leaks in applications.
- Large cache usage (usually normal).
- Too many processes or containers.
- High load causing more memory usage.
- Misconfigured services.
- Insufficient swap space.

8 Common Beginner Mistakes

- Assuming low free memory means out of memory.
- Ignoring the available field.
- Not checking swap usage.
- Not identifying the process using memory.
- Restarting services without finding the real cause.



Key Takeaways

- Use available instead of free.
- Check cache, swap and running processes.
- Look for evidence before taking action.
- Use multiple tools to confirm the problem.
- Understand the normal behaviour of Linux memory.



Practice This

- 1 Check memory usage with free -h.
- 2 Monitor real time usage with top and vmstat.
- 3 Find memory-heavy processes with ps.
- 4 View detailed memory info in /proc/meminfo.
- 5 Trigger a test memory load and observe the changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Why Is CPU Usage So High?



Why This Matters?

High CPU usage can make applications slow, cause service failures and affect the entire server. You need to understand what CPU usage means, learn how to identify the process responsible and choose the right action.



Junior Engineer Tip

High CPU usage is a symptom, not a problem by itself. Always find the cause before taking action.

1 Load Average vs CPU Utilization

They are different and mean different things.

Metric	What It Means	Command
CPU utilization	Percentage of CPU currently in use	top, ps
Load average	Number of tasks waiting for CPU (runnable or in uninterruptible sleep)	uptime
Both together	High load with high CPU means CPU is busy. High load with low CPU may indicate I/O or other issues.	uptime, top, vmstat

2 Check System Load and Uptime

See current load average and how long the system has been running.

```
ubuntu@linux:~$ uptime
```

```
10:24:15 up 5 days, 2:13, 3 users, load average: 4.25, 3.10, 2.01
```

- Load average shows the number of tasks waiting for CPU.
- Three values: 1 minute, 5 minutes and 15 minutes.
- Compare load average with the number of CPU cores (using nproc).
- A load higher than the number of CPU cores often means the server is busy.

3 Find the Process Using the Most CPU

Use top to find the process in real time.

```
ubuntu@linux:~$ top
```

```
PID  USER  PR  NI  VIRT  RES  %CPU  %MEM  COMMAND
2345  root   20   0  162m  58m  85.3   1.2   python
5678  ubuntu 20   0  254m  120m 45.1   2.5   nginx
9012  ubuntu 20   0   90m   40m 12.3   0.8   java
```

- Sort by %CPU (press P in top).
- Identify the PID, user and command.
- Check if the process is expected or unusual.
- Observe if it is a single process or multiple processes.

4 Use ps for Detailed Information

Get more details about a specific process.

```
ubuntu@linux:~$ ps aux --sort=-%cpu | head -n 10
```

```
USER  PID  %CPU  %MEM  VSZ   RSS  COMMAND
root   2345  85.3   1.2  162000 60000  python
ubuntu 5678  45.1   2.5  254000 120000  nginx
ubuntu 9012  12.3   0.8   90000 40000   java
```

```
ubuntu@linux:~$ ps -p 2345 -o pid,ppid,user,%cpu,%mem,cmd
```

```
PID  PPID  USER  %CPU  %MEM  CMD
2345   1   root   85.3   1.2   python app.py
```

- Use `ps aux --sort=-%cpu` to find top CPU consumers.
- Use `ps -p <PID> -o...` to see detailed information.
- Check the parent process (PPID) and command.

5 Use vmstat to See System Activity

Shows CPU, memory, I/O and process information.

```
ubuntu@linux:~$ vmstat 1 5
```

```
procs -----memory----- --swap-- --io-- --system-- -----cpu-----
r b swpd free free buff cache s  d o bo in cs us sy id wa st
5 0 0 502000 320000 1024000 0 0 150 320 2000 4500 75 10 15 0 0
3 0 0 498000 318000 1020000 0 0 120 200 1800 3900 60 8 32 0 0
1 0 0 510000 330000 1030000 0 0 100 150 1600 3100 25 5 70 0 0
```

- r: number of runnable processes waiting for CPU.
- us: CPU time used by user processes.
- sy: CPU time used by kernel.
- id: idle time (low value means high CPU usage).
- wa: time waiting for I/O (high wa can indicate I/O issues).

6 Identify and Investigate the Cause

Check what the process is doing and why it is using CPU.

```
ubuntu@linux:~$ ps -p 2345 -o pid,user,cmd
```

```
PID  USER  CMD
2345  root   python app.py
```

```
ubuntu@linux:~$ ls -l /proc/2345/exe
```

```
lrwxrwxrwx 1 root root 0 Oct 01 10:24 /proc/2345/exe -> /usr/bin/python3.10
```

- Check the full command and path.
- Look at application logs.
- Determine if the high CPU usage is expected.
- If it is unexpected, check for bugs, loops or misconfiguration.
- Decide the right action (restart, fix, or stop the process).

7 Common Causes of High CPU Usage

- Runaway processes or infinite loops.
- Application bugs or memory leaks.
- High traffic or load on the application.
- Malware or unauthorized processes.
- Scheduled jobs or cron tasks.
- Resource intensive operations (e.g. compression).
- Misconfigured services or containers.

8 Common Beginner Mistakes

- Killing a process without understanding it.
- Ignoring the real cause.
- Confusing load average with CPU usage.
- Not checking all processes (only looking at the top one).
- Not checking if the process is expected.
- Not reviewing logs before taking action.
- Restarting services without fixing the cause.



Key Takeaways

High CPU usage affects performance and stability. Use top, ps, vmstat and uptime to investigate. Find the real process and understand the cause. Don't take action blindly. Fix the root cause to prevent it from happening again.



Practice This

- 1 Check CPU usage with top and uptime.
- 2 Find the top 3 processes using CPU.
- 3 Investigate one process in detail with ps.
- 4 Use vmstat to observe system activity.
- 5 Identify the cause and decide the right action.

Is the Network Actually Down?



Why This Matters?

Network issues can make applications unavailable, break communication between services, and impact users. Troubleshooting by layer helps you find the real cause quickly instead of just saying "the network is down."



Junior Engineer Tip

The network is rarely just down. Check each layer (step by step) to find exactly where the problem is.

1 Check Network Interfaces with ip addr

View network interfaces and IP addresses.

```
ubuntu@linux:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP>
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 192.168.1.10/24 brd 192.168.1.255
```

- Shows all network interfaces (lo, eth0, etc).
- Check if the interface is UP.
- Verify the IP address and subnet.
- Look for errors or a missing IP.

2 Check Routes and Default Gateway

See routing table and default gateway.

```
ubuntu@linux:~$ ip route show
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link
10.0.0.0/16 via 192.168.1.1 dev eth0
```

- default shows the gateway.
- Check that the correct interface is used.
- Verify routes to other networks.
- Missing or wrong default route can cause no internet connectivity.

3 Test Connectivity with ping

Check basic network connectivity.

```
ubuntu@linux:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=12.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=11.8 ms
```

- Tests if a host is reachable.
- Use `-c` to send a specific number of packets.
- Try local gateway, a public IP (8.8.8.8) and a domain.
- If ping fails, check the interface, routes or firewall.

4 Check Open Connections with ss

View active network connections and listening ports.

```
ubuntu@linux:~$ ss -tulnp
Netid  State  Local Address:Port  Peer Address:Port
tcp    LISTEN  0.0.0.0:22           0.0.0.0:*
tcp    LISTEN  127.0.0.1:3306      0.0.0.0:*
udp    UNCONN  0.0.0.0:53         0.0.0.0:*
```

- `-t` shows TCP, `-u` shows UDP, `-l` shows listening ports, `-n` shows numeric addresses, `-p` shows process.
- Use this to confirm if a service is listening.
- Helps verify if connections are established.

5 Test Web Services with curl

Check if you can reach a web service.

```
ubuntu@linux:~$ curl -I https://www.google.com
HTTP/2 200
content-type: text/html; charset=UTF-8
server: gws
content-length: 173531
```

- Checks connectivity to a web server.
- `-I` shows only headers (faster).
- Use http or https depending on the service.
- Helps identify if the issue is network, DNS or application.

6 Check DNS Resolution with dig

Test DNS name resolution.

```
ubuntu@linux:~$ dig google.com
;; ANSWER SECTION:
google.com.      300    IN     A      142.250.190.78
google.com.      300    IN     A      142.250.190.110
```

- Converts a domain name to an IP address.
- Confirms if DNS is working.
- If this fails, check your DNS configuration.
- Use `dig` or `resolvectl` for more details.

7 Check DNS Configuration with resolvectl

View current DNS settings.

```
ubuntu@linux:~$ resolvectl status
Global
DNS Servers: 8.8.8.8 1.1.1.1
DNS Domain: ~
Link 2 (eth0)
Current DNS Server: 8.8.8.8
DNS Servers: 8.8.8.8 1.1.1.1
```

- Shows configured DNS servers.
- Verify you are using the correct DNS.
- Helps troubleshoot DNS related issues.

8 Troubleshoot by Layer

Follow this order to find the issue.

- 1 Check interface is UP and has an IP.
- 2 Check routes and default gateway.
- 3 Test connectivity with ping.
- 4 Check DNS resolution with dig or resolvectl.
- 5 Test the actual service with curl.
- 6 Check listening ports with ss.
- 7 Review firewall rules if needed.
- 8 Identify the layer where it fails and fix only that issue.



Key Takeaways

Do not assume the network is down.

- Troubleshoot by layer: interface → route → connectivity → DNS → service.
- Use the right tools for each layer.
- Look for evidence and fix the real cause.



Practice This

- 1 Find your IP address and default gateway.
- 2 Ping your gateway, 8.8.8.8 and a domain name.
- 3 Use dig to resolve a domain.
- 4 Use curl to check a website.
- 5 Use ss to check listening ports.
- 6 Identify and fix a network issue in a test environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Why Can't I Reach This Port?



Why This Matters?

You may not be able to reach a port because the application is not listening, it is bound to the wrong address, a firewall is blocking the traffic, or there is a network issue. Troubleshooting step by step helps you find the real cause.



Junior Engineer Tip

A "connection refused" usually means nothing is listening on the port. A timeout often means a firewall or network issue.

1 Check Listening Ports with ss -lntp

See which ports are open and which process is listening.

```
ubuntu@linux:~$ ss -lntp
State      Local Address:Port  Peer Address:Port Process
LISTEN     0.0.0.0:22          0.0.0.0:*        users:("sshd",pid=1234)
LISTEN     127.0.0.1:3306      0.0.0.0:*        users:("mysqld",pid=5678)
LISTEN     [::]:80            [::]:*          users:("nginx",pid=9012)
```

- -l shows only listening sockets.
- -n shows numeric addresses and ports.
- -t shows TCP sockets.
- -p shows the process using the port.
- Check if the service is listening on the expected address and port.

2 Test the Port Locally with curl

Check if the port works on the local server.

```
ubuntu@linux:~$ curl -v http://localhost:8080/
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET / HTTP/1.1
> Host: localhost:8080
>
< HTTP/1.1 200 OK
< Server: nginx
```

- Use localhost to test the service on the same server.
- If this works locally, the issue is likely network, firewall or remote access.
- If it fails, the application may not be running or listening on the correct address.

3 Test the Port from a Remote Server

Check if the port is reachable from another machine.

```
ubuntu@remote:~$ curl -v http://192.168.1.10:8080/
* Trying 192.168.1.10:8080...
* Connected to 192.168.1.10 (192.168.1.10) port 8080 (#0)
> GET / HTTP/1.1
> Host: 192.168.1.10:8080
>
< HTTP/1.1 200 OK
```

- Replace the IP and port with your target.
- If it succeeds, the port is reachable.
- If it fails, check firewall rules, network routes or the application state.

4 Understand Common Error Messages

Different errors mean different problems.

Error Message	What It Means	Common Causes
Connection refused	The target host is reachable but nothing is listening on the port.	Application not running, wrong port, or listening on a different address (e.g. 127.0.0.1 only).
Connection timed out	No response from the target host.	Firewall blocking, network issue, or host down.
No route to host	No network route to the target.	Routing issue or wrong IP address.
Host unreachable	The host cannot be reached.	Network or gateway issue.

5 Check Firewall Rules

Make sure the firewall allows the port.

For UFW (Ubuntu)

```
ubuntu@linux:~$ sudo ufw status
Status: active

To Action From
22/tcp ALLOW Anywhere
8080/tcp ALLOW Anywhere
```

For firewalld (RHEL/CentOS)

```
ubuntu@linux:~$ sudo firewall-cmd
--list-ports
22/tcp 80/tcp 443/tcp
ubuntu@linux:~$ sudo firewall-cmd
--add-port=8080/tcp --permanent
success
```

- Check if the port is allowed in the firewall.
- Open the port if needed and test again.
- Also check cloud security groups if using a cloud server.

6 Check Application State and Binding

Ensure the application is running and listening on the right address.

```
ubuntu@linux:~$ sudo systemctl status nginx
● nginx.service - A high performance web server
Active: active (running) since ...
```

```
ubuntu@linux:~$ ss -lntp | grep 8080
LISTEN 0.0.0.0:8080 0.0.0.0:* users:(("python",pid=2345))
```

- Confirm the application service is running.
- Check which address it is bound to (0.0.0.0, 127.0.0.1 or a specific IP).
- If it is bound to 127.0.0.1, it will only accept local connections.
- Verify the correct port and configuration file.

7 Local vs Remote Binding

The address an application listens on matters.

Address	What It Means
127.0.0.1	Only accepts local connections (on the same server).
0.0.0.0	Accepts connections from any IP (on all interfaces).
192.168.1.10	Only accepts connections on this specific IP.
[::]	IPv6 equivalent of 0.0.0.0.

8 Troubleshoot by Layer

Check each layer to find the issue.

- 1 Is the application running?
- 2 Is it listening on the correct port and address?
- 3 Does it work locally (localhost)?
- 4 Is the firewall allowing the port?
- 5 Can you reach it from a remote server?
- 6 Are there any network or routing issues?
- 7 Check logs for errors.
- 8 Fix the real cause and test again.



Key Takeaways

Connection refused usually means nothing is listening on the port.
A timeout usually means a firewall or network issue.
Check the application, firewall, network and configuration.
Troubleshoot by layer instead of guessing.



Practice This

- 1 Find listening ports on your server.
- 2 Test a local service with curl.
- 3 Test the same port from a remote server.
- 4 Create a simple service and change the bind address.
- 5 Simulate a firewall block and observe the error.

SSH Access & Authentication Failures

Why This Matters?

SSH is used for secure remote administration. Authentication failures can prevent you from accessing servers, delay incident response and affect operations. Understanding the SSH flow and troubleshooting systematically helps you quickly find and fix the real cause.



Junior Engineer Tip

"Permission denied (publickey)" usually means the server did not accept your key. Check the key, permissions and server configuration step by step.

1 Understand the SSH Flow

Know what happens when you run `ssh user@server`.



- Client connects to the server on port 22.
- Server sends its host key.
- Client verifies the host key.
- Client authenticates using a key or password.
- If authentication is successful, you get a shell.

2 Check Your SSH Keys Locally

Verify that your keys exist and are correct.

```
ubuntu@linux:~$ ls -l ~/.ssh
total 16
-rw----- 1 ubuntu ubuntu 2602 Oct 1 10:20 id_rsa
-rw-r--r-- 1 ubuntu ubuntu 564 Oct 1 10:20 id_rsa.pub
-rw-r--r-- 1 ubuntu ubuntu 231 Oct 1 10:25 known_hosts
```

- Make sure your private key (e.g. `id_rsa`) exists.
- Check the permissions (private key should be 600).
- Verify you are using the correct key with the `-i` option.
- Confirm the public key matches the one on the server.

3 Test Connection with Verbose Mode

Use `ssh -v` to see detailed logs.

```
ubuntu@linux:~$ ssh -v -i ~/.ssh/id_rsa user@192.168.1.10
OpenSSH 9.6p1, OpenSSL 3.0.2
debug1: Reading configuration data /etc/ssh/ssh_config
debug1: Connecting to 192.168.1.10 [192.168.1.10] port 22.
debug1: Offering public key: /home/ubuntu/.ssh/id_rsa
debug1: Server accepts key: pkalg ssh-ed25519
debug1: Authentication succeeded (publickey).
```

- Look for which key is being used.
- Check if the server accepts or rejects the key.
- Identify the exact step where authentication fails.
- Use this output to guide your next check.

4 Check Authorized Keys on the Server

Verify that your public key is added correctly.

```
ubuntu@server:~$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJuy... ubuntu@laptop
```

- Make sure your public key is in `~/.ssh/authorized_keys`.
- Check file and directory permissions.

Correct Permissions

```
~/.ssh 700
~/.ssh/authorized_keys 600
```

Set Permissions (if needed)

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

5 Check SSH Server Configuration

Verify that the server allows key authentication.

```
ubuntu@server:~$ sudo grep -E 'PasswordAuthentication|
PubkeyAuthentication' /etc/ssh/sshd_config
PubkeyAuthentication yes
PasswordAuthentication no
```

- PubkeyAuthentication should be yes.
- PasswordAuthentication can be yes or no (depending on your policy).
- After changes, restart SSH: `sudo systemctl restart sshd`.
- Check for Match blocks that may override settings.

6 Check Server Logs for the Real Cause

The logs show why the authentication failed.

```
ubuntu@server:~$ sudo tail -n 20 /var/log/auth.log
Oct 1 10:32:15 server sshd[1234]: Failed publickey for
ubuntu from 192.168.1.20 port 54321 ssh2: RSA SHA256:...
Oct 1 10:32:15 server sshd[1234]: Connection closed
by authenticating user ubuntu 192.168.1.20 port 54321
```

- Look for "Failed publickey" or "Permission denied" messages.
- Check the username, IP address and key type.
- This helps you confirm whether the server is receiving your key.
- Also check `/var/log/secure` on RHEL/CentOS systems.

7 Common Causes of Permission Denied

- Public key not in `authorized_keys`.
- Wrong file or directory permissions.
- Using the wrong key or username.
- Server not allowing public key authentication.
- Key not added to SSH agent (if using agent).
- Match block in `sshd_config` overriding settings.
- Connecting to the wrong server or port.
- Account locked or restricted on the server.

8 Troubleshoot Step by Step

- 1 Confirm you can reach the server (ping, port 22).
- 2 Check you are using the correct key.
- 3 Verify local key permissions.
- 4 Check server `authorized_keys` and permissions.
- 5 Review `sshd_config` settings.
- 6 Check server logs for the error message.
- 7 Test again with `ssh -v`.
- 8 Fix the issue and confirm you can log in.



Key Takeaways

SSH uses keys or passwords to authenticate. Permission denied (publickey) means the server did not accept your key. Use `ssh -v` and server logs to find the real cause. Always follow secure permission settings. Troubleshoot step by step, do not guess.



Practice This

- 1 Generate a new SSH key pair.
- 2 Add your public key to a test server.
- 3 Configure permissions correctly.
- 4 Test the connection with `ssh -v`.
- 5 Intentionally break a permission and fix it.
- 6 Check the server logs and identify the error.

Packages, Updates & Safe Changes

Why This Matters?

Packages and updates keep your system secure, stable and feature rich. Understanding how package managers work, the difference between update and upgrade, and making safe changes helps you avoid breaking the system and ensures you can roll back if needed.



Junior Engineer Tip

Always know what you are installing or updating. Read package information, check changes and have a backup plan before making system changes.

1 Common Package Managers

Different Linux distributions use different package managers.

Distribution	Package Manager	Common Commands
Ubuntu / Debian	apt	apt update, apt install
RHEL / CentOS / Rocky	dnf	dnf install, dnf update
Amazon Linux	dnf / yum	dnf install, dnf update
SUSE	zypper	zypper install, zypper update

- Package managers download and install software from repositories.
- Packages include application files, dependencies and metadata.
- Repositories can be official (trusted) or third party.

2 Update vs Upgrade

Update refreshes package metadata. Upgrade installs newer versions.

Ubuntu / Debian (apt)

```
ubuntu@linux:~$ sudo apt update
ubuntu@linux:~$ sudo apt upgrade
```

- apt update: refreshes package lists from repositories.
- apt upgrade: installs newer versions of installed packages.
- apt full-upgrade: also handles dependency changes (bigger upgrades).

RHEL / CentOS (dnf)

```
ubuntu@linux:~$ sudo dnf makecache
ubuntu@linux:~$ sudo dnf upgrade
```

- dnf makecache: refreshes package metadata.
- dnf upgrade: installs newer versions of installed packages.
- dnf distro-sync: keeps system in sync with the repository (may downgrade).

3 Search and Install Packages

Find package information and install safely.

```
ubuntu@linux:~$ apt search nginx
ubuntu@linux:~$ apt show nginx
ubuntu@linux:~$ sudo apt install nginx

ubuntu@linux:~$ dnf search nginx
ubuntu@linux:~$ dnf info nginx
ubuntu@linux:~$ sudo dnf install nginx
```

- search: find available packages.
- show/info: view package details (version, size, description).
- install: download and install the package with dependencies.

4 Check Installed Packages

Verify what is installed and its version.

Ubuntu / Debian (apt)

```
ubuntu@linux:~$ dpkg -l nginx
ubuntu@linux:~$ apt list
--installed | grep nginx
```

- dpkg -l: show installed packages.
- apt list --installed: list installed packages.
- apt show <package>: package information.

RHEL / CentOS (dnf)

```
ubuntu@linux:~$ dnf list
installed | grep nginx
ubuntu@linux:~$ dnf info nginx
```

- dnf list installed: show installed packages.
- dnf info <package>: package information.
- rpm -q <package>: check if a package is installed.

5 Package Metadata and Repositories

Check repository information and package metadata.

```
ubuntu@linux:~$ apt policy nginx
ubuntu@linux:~$ apt source nginx

ubuntu@linux:~$ dnf repolist
ubuntu@linux:~$ dnf list nginx --showduplicates
```

- policy: shows available versions and repositories.
- source: downloads the source package.
- repolist: lists configured repositories (RHEL/CentOS).
- --showduplicates: shows all available versions.

6 Make Configuration Changes Safely

Backup configuration files before changing them.

```
ubuntu@linux:~$ sudo cp /etc/nginx/nginx.conf \
/etc/nginx/nginx.conf.bak
ubuntu@linux:~$ sudo nano /etc/nginx/nginx.conf
ubuntu@linux:~$ sudo systemctl restart nginx
ubuntu@linux:~$ sudo systemctl status nginx
```

- Always backup configuration files before editing.
- Make small changes and test.
- Use a text editor (nano or vim).
- After changes, restart or reload the service.
- Verify the service is running and working as expected.

7 Maintenance Caution

- Read package notes and changelogs.
- Check if the update requires a reboot.
- Avoid updating critical servers during peak hours.
- Test updates in a non production environment when possible.
- Have a rollback plan (previous package version).
- Monitor services after the update.

8 Common Mistakes

- Running upgrade without reading changes.
- Not checking service compatibility.
- Updating during peak traffic.
- Ignoring dependency changes.
- Not having a backup or rollback plan.
- Installing packages from untrusted sources.
- Forgetting to verify that services still work after the update.



Key Takeaways

- Understand your package manager.
- Know the difference between update and upgrade.
- Check package information before installing.
- Backup configuration files.
- Make changes carefully and verify afterward.



Practice This

- 1 Find and install a package on your system.
- 2 Check the installed version.
- 3 Backup a configuration file and make a change.
- 4 Update a package and verify the service.
- 5 Find all available versions of a package.

The Linux Incident Survival Drill

Scenario

A user reports: "Our application is unavailable."
Follow a systematic approach to find the real cause,
apply the smallest safe fix and verify the solution.



Junior Engineer Tip

Do not guess. Collect evidence at each layer, move step by step and fix only what is necessary.

1 Confirm the Problem

reproduce the issue and understand the impact.

```
ubuntu@linux:~$ curl -v http://app.example.com
* Trying 10.0.0.10:80...
* connect: Connection refused
curl: (7) Failed to connect to app.example.com port 80
```

- Check if the issue is external or internal.
- Note the exact error message.
- Confirm who is affected and since when.

2 Check the Host Status

Make sure the server is up and reachable.

```
ubuntu@linux:~$ uptime
10:24:15 up 5 days, 3:12, 2 users, load average: 0.10, 0.20, 0.15
ubuntu@linux:~$ hostname -I
10.0.0.10 192.168.1.10
```

- Verify the server is running.
- Check load average and basic health.
- Confirm you are on the correct server.

3 Check the Service

Verify the application service is running.

```
ubuntu@linux:~$ sudo systemctl status nginx
● nginx.service - A high performance web server
Active: inactive (dead) since Oct 01 10:15:22
```

- Check service status (active or inactive).
- Review recent service logs.
- Start or restart if needed.

4 Check the Process

Confirm the process is running.

```
ubuntu@linux:~$ ps -ef | grep nginx
root      2345      1  0 10:15 ?        00:00:00 nginx: master process
www-data  2346    2345  0 10:15 ?        00:00:00 nginx: worker process
```

- Verify the process exists.
- Check for abnormal CPU or memory usage.
- If not running, check logs for the reason.

5 Check the Application Logs

Look for errors around the time of the incident.

```
ubuntu@linux:~$ sudo journalctl -u nginx -n 10
Oct 01 10:14:55 linux nginx[2345]: bind() to 0.0.0.0:80
failed (98: Address already in use)
```

- Check service logs with `journalctl`.
- Look for errors, warnings or clues.
- Identify the real cause from log messages.

6 Check System Resources

Ensure CPU, memory and disk are not causing the issue.

```
ubuntu@linux:~$ free -h
               total    used    free   shared  buff/cache   available
Mem:            7.7G    6.1G    500M    120M     1.1G        1.2G
Swap:           2.0G    200M    1.8G
```

- Check CPU with `top` or `vmstat`.
- Check memory with `free -h`.
- Check disk space with `df -h` and inodes with `df -i`.

7 Check Network and Port

verify the port is open and reachable.

```
ubuntu@linux:~$ ss -lntp | grep :80
LISTEN 0      511 0.0.0.0:80      0.0.0.0:*    users:(("nginx",pid=2345)
ubuntu@linux:~$ sudo firewall-cmd --list-ports
80/tcp 22/tcp
```

- Check if the service is listening on the expected port.
- Verify firewall rules allow the port.
- Test from local and remote using `curl` or `ping`.

8 Check Permissions

Ensure files, directories and configs have correct permissions.

```
ubuntu@linux:~$ ls -l /etc/nginx/nginx.conf
-rw-r--r-- 1 root root 1234 Oct 01 09:50 /etc/nginx/nginx.conf
ubuntu@linux:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

- Check file and directory permissions.
- Validate configuration using the application's test command.
- Fix ownership or permissions if misconfigured.

9 Identify the Root Cause

Use the evidence to find the actual cause.

Example root cause:

Nginx failed to start because port 80 was already in use by another process.

- Combine information from logs, process, port and config.
- Confirm the exact reason, not just the symptoms.
- Document the root cause.

10 Apply the Smallest Safe Fix

Make a targeted change to resolve the issue.

```
ubuntu@linux:~$ sudo systemctl stop apache2
ubuntu@linux:~$ sudo systemctl start nginx
ubuntu@linux:~$ sudo systemctl status nginx | head -n 3
● nginx.service - A high performance web server
Active: active (running) since Oct 01 10:25:10
● Stop or reconfigure the conflicting service.
● Apply the minimum change needed.
● Avoid unnecessary changes.
```

11 Verify the Solution

Confirm the application is working as expected.

```
ubuntu@linux:~$ curl -I http://app.example.com
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 01 Oct 2026 10:27:15 GMT
```

- Test from local and remote clients.
- Confirm the service is stable.
- Monitor for a few minutes to ensure no new errors.

12 Document the Incident

Record what happened, the cause and the fix.

- Time of the incident.
- What was affected.
- Steps you took to troubleshoot.
- Root cause.
- The fix you applied.
- How you verified the solution.
- Lessons learned and prevention ideas.

Key Takeaways

- Follow a structured approach.
- Check each layer systematically.
- Use real evidence from the system.
- Fix only what is necessary.
- Verify and document everything.



Practice This

- Recreate a similar incident in a lab.
- Break the application intentionally.
- Follow all the troubleshooting steps.
- Find the root cause and fix it.
- Document the incident.